PARCIAL PRÁCTICO · SEGUNDO TERCIO

Evolución de ToDo: gestión de categorías

El reto: implementar una funcionalidad nueva en la solución Full Stack construida durante el laboratorio.

Asignatura: Desarrollo y Operaciones de Software (DOSW)

Modalidad: trabajo sobre el repositorio y el código existente de la aplicación ToDo.

Proyecto base: Lab6-G4-AppToDo / README.md

Nombre(s): _____

Equipo / grupo: _____ Fecha: _____

1. Contexto

En el laboratorio anterior se desarrolló una aplicación ToDo que permite crear, consultar, modificar, eliminar y cambiar el estado de tareas. La solución utiliza una API REST en Spring Boot, persistencia JPA con PostgreSQL ejecutado en Docker y una interfaz en React. La vista de calendario semanal figuraba como un bono del laboratorio anterior.

La nueva necesidad surge porque el usuario tiene tareas de diferentes ámbitos (estudio, trabajo y asuntos personales), pero no cuenta con una forma consistente de agruparlas y filtrarlas mediante categorías configurables.

2. Objetivo del parcial

Analizar la nueva necesidad, planearla en Jira, actualizar la arquitectura y los modelos de diseño, implementar la funcionalidad de extremo a extremo, automatizar sus pruebas y documentar la evolución del sistema en el README.

Qué se espera al terminar

- Una funcionalidad operativa desde React que persista sus datos en PostgreSQL.
- Épica, Feature, historias de usuario y tareas documentadas y trazables en Jira.
- Diagramas C4 y de clases actualizados en el README, junto con evidencias y un video local.

Importante: las pantallas de referencia son mockups de baja fidelidad. No constituyen un diseño visual obligatorio; los estudiantes deben diseñar la solución técnica y sus historias de usuario.

3. Requerimiento funcional: categorías

Agregar al sistema ToDo la capacidad de administrar categorías y asociar una categoría a cada tarea. Una tarea puede quedar sin categoría, y las tareas registradas antes del parcial deben conservarse.

Código	Comportamiento esperado
RF-01	Crear una categoría con nombre y color seleccionable de una paleta de cuatro opciones.
RF-02	Consultar las categorías existentes, mostrando su nombre y color.
RF-03	Modificar el nombre y el color de una categoría existente.
RF-04	Eliminar una categoría únicamente si no tiene tareas asociadas; en otro caso, informar el conflicto sin eliminarla.
RF-05	Asignar o cambiar la categoría al crear o editar una tarea. La categoría es opcional.
RF-06	Filtrar el listado de tareas por una categoría, por todas las categorías o por tareas sin categoría.
RF-07	Mantener intacto el CRUD original, los estados y prioridades ya existentes, y conservar los registros previos.

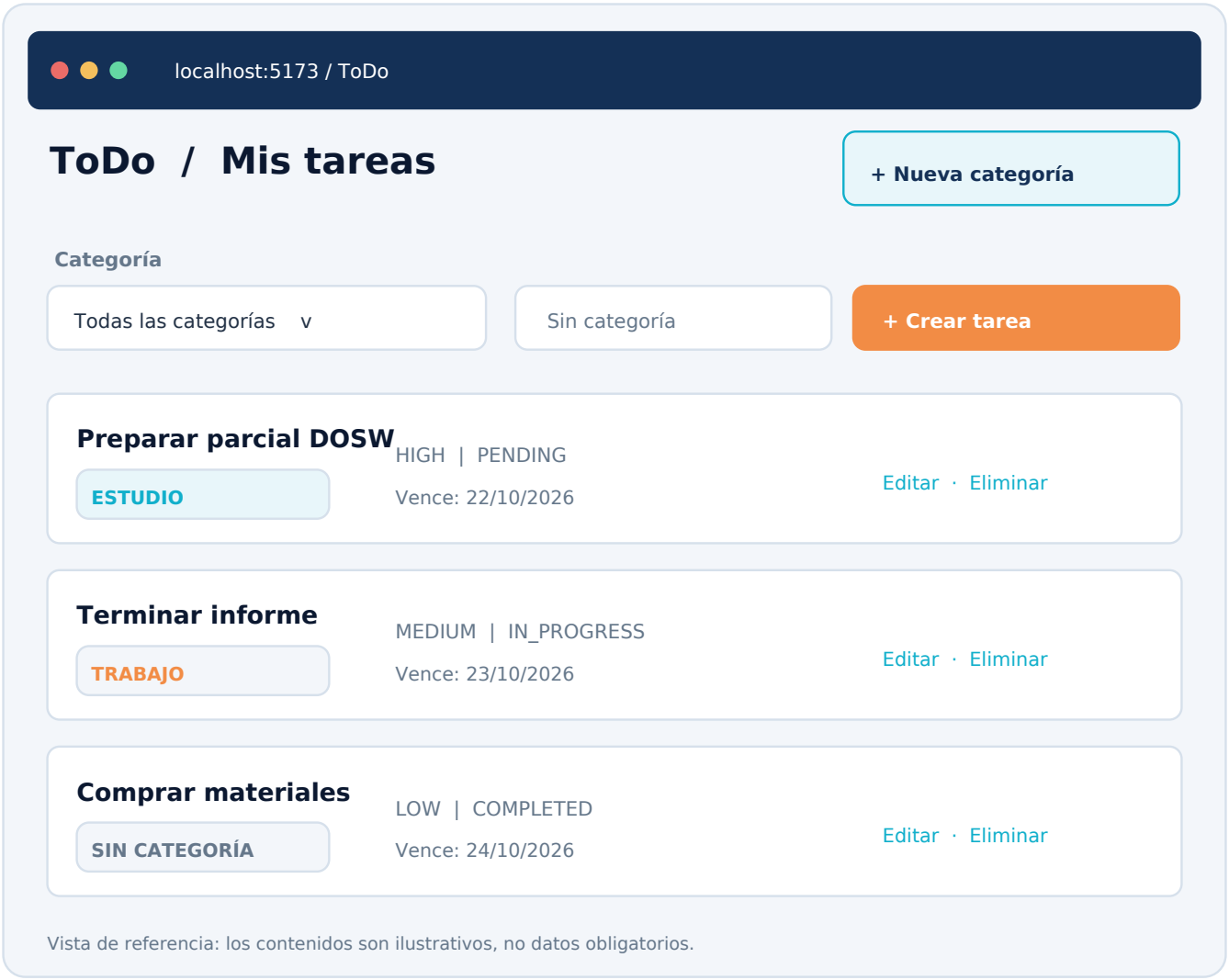
4. Reglas de negocio y validaciones

- El nombre de categoría es obligatorio, tendrá entre 3 y 40 caracteres y no podrá repetirse ignorando diferencias entre mayúsculas y minúsculas ni espacios al inicio o al final.
- El color de la categoría debe elegirse entre las cuatro opciones del prototipo; el equipo define cómo representarlo en el modelo de datos.
- Cada tarea tendrá cero o una categoría. Una misma categoría podrá agrupar muchas tareas.
- Los registros de tareas que ya existen se conservarán inicialmente sin categoría; no deben borrarse ni alterarse por la migración.
- El filtro debe obtener los resultados desde la API REST; no se acepta usar arreglos estáticos ni depender exclusivamente de datos simulados en React.
- Si se intenta eliminar una categoría con tareas asociadas, la API deberá responder con 409 Conflict y el front-end deberá mostrar un mensaje comprensible.
- Las validaciones deben aplicarse en el back-end; el front-end puede añadir validación preventiva.

> **No se solicita implementar autenticación, despliegue, CI/CD ni nuevas reglas de seguridad para esta evaluación.**

5. Prototipo de baja fidelidad · Vista principal

Pantalla de referencia A. Se incorpora el filtro por categoría al listado de tareas y el acceso para administrar categorías. La navegación y la distribución exacta son decisión del equipo.

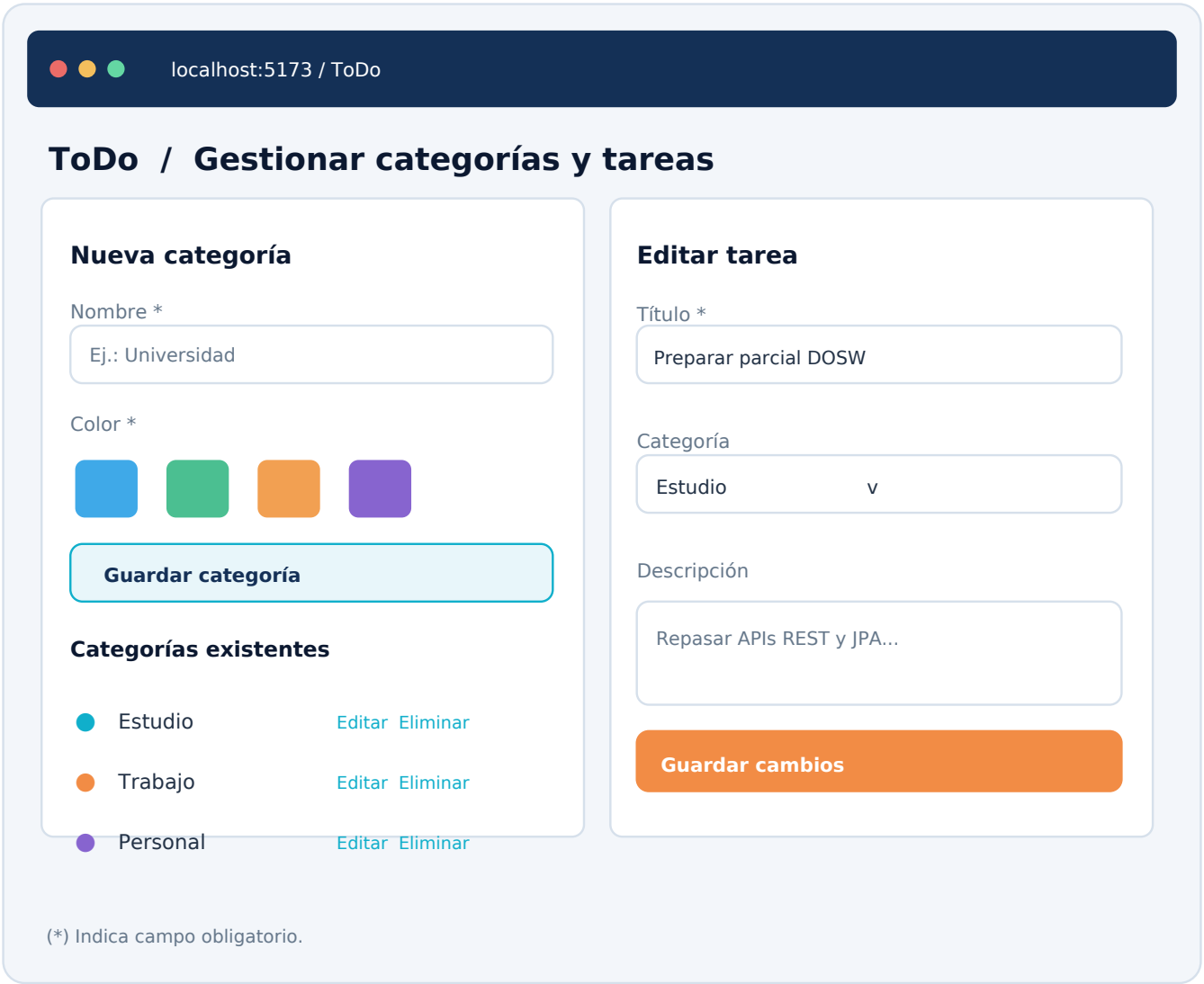


Interacciones que debe resolver el estudiante

- Al seleccionar Todas las categorías, se muestran todas las tareas.
- Al seleccionar una categoría específica, la lista recupera únicamente las tareas asociadas.
- Al seleccionar Sin categoría, se consultan y muestran las tareas sin asociación.
- Después de guardar o modificar una categoría o tarea, los filtros y listados reflejan la información actualizada.

6. Prototipo de baja fidelidad · Formularios

Pantalla de referencia B. Administración de categorías y actualización de una tarea. La representación no obliga a mostrar ambos formularios al mismo tiempo.



Comportamientos y estados de interfaz

- Incluir acciones para crear, editar y eliminar categorías, así como seleccionar su color.
- Agregar el selector de categoría al formulario de crear y editar tarea, incluyendo la opción sin categoría.
- Mostrar mensajes útiles para errores de validación, duplicidad de nombres y conflicto al intentar eliminar una categoría en uso.
- No alterar las acciones existentes de editar, cambiar estado o eliminar tareas.

7. Trabajo que deben desarrollar

A. Análisis, backlog y Jira

- Crear en Jira una Épica que represente la evolución funcional; definir una Feature asociada y redactar las historias de usuario necesarias.
- Cada historia debe contener descripción en formato Como / Quiero / Para y criterios de aceptación verificables (preferiblemente Dado / Cuando / Entonces).
- Descomponer las historias en tareas técnicas de diseño, persistencia, back-end, front-end, pruebas y documentación. Dejar responsables, estados y trazabilidad.
- Si el proyecto Jira no dispone del tipo de incidencia Feature, representarla mediante un issue identificado como Feature y explicar el vínculo con la épica y las historias.

B. Arquitectura y diseño

- Actualizar los diagramas C4 nivel 1 (contexto), nivel 2 (contenedores) y nivel 3 (componentes). Reflejar los cambios relevantes de esta funcionalidad y justificar cuando un nivel no cambie estructuralmente.
- Actualizar el diagrama de clases del dominio y las relaciones necesarias para representar categorías y tareas.
- Diseñar y registrar la evolución del esquema de PostgreSQL y del mapeo JPA, preservando los datos previos. El estudiante decide las entidades, restricciones y clases necesarias.

C. Implementación Full Stack

- Implementar persistencia en PostgreSQL con Spring Data JPA, sin sustituirla por memoria o datos estáticos.
- Incorporar las capas pertinentes de Controller, Service, Repository, DTO y manejo de excepciones para la funcionalidad.
- Diseñar recursos y contratos REST coherentes con el dominio, incluidos validaciones y códigos HTTP adecuados.
- Actualizar la interfaz React, consumir los nuevos recursos REST y mostrar estados de carga, éxito y error.
- Conservar el comportamiento previo del CRUD; la nueva funcionalidad debe integrarse a la aplicación existente.

D. Calidad y evidencia

- Agregar pruebas unitarias de las reglas de categorías y de los componentes React afectados; incluir al menos una prueba del contrato REST y una de persistencia o integración.
- Verificar casos normales y de error: nombre duplicado, categoría inexistente, eliminación con tareas asociadas y filtro sin resultados.
- Mantener una evidencia de pruebas ejecutadas y del funcionamiento desde la interfaz.

8. Entregables y evidencias obligatorias

A. README del repositorio

Actualizar el README.md del repositorio utilizado para el parcial e incluir, como mínimo:

- Descripción y alcance de la funcionalidad implementada; historias de usuario y criterios de aceptación.
- Enlaces o identificadores de la Épica, Feature, historias y tareas en Jira y evidencia de su trazabilidad.
- Diagramas C4 actualizados (contexto, contenedores, componentes) y diagrama de clases actualizado; deben visualizarse en el README como imágenes o diagramas renderizados.
- Cambios del esquema de base de datos; resumen de los endpoints implementados y pruebas realizadas.
- Instrucciones para levantar y probar localmente la aplicación y enlace al video de demostración.

B. Video del funcionamiento local

Entregar un enlace accesible a un video de 5 a 8 minutos en el que se vea la aplicación ejecutándose localmente, con React, Spring Boot y PostgreSQL en Docker. Debe demostrar:

- La creación, consulta, edición y eliminación de una categoría, incluyendo el rechazo de eliminación cuando tiene tareas asociadas.
- La asociación de una categoría a una tarea y el cambio o retiro de la asociación.
- El filtro por categoría, la opción Todas y la opción Sin categoría.
- La persistencia real y la conservación del CRUD anterior; la ejecución y resultado de las pruebas automatizadas.

C. Evidencia de entrega

Entregar el enlace al repositorio con el código actualizado, el README, la documentación de arquitectura, los cambios de base de datos y el video. El evaluador debe poder identificar qué se cambió y cómo se comprobó.

9. Rúbrica de evaluación

Criterio	Puntaje
Análisis, historias y trazabilidad en Jira	0,80
Diagramas C4 y diagrama de clases	0,85
Modelo de datos y persistencia JPA/PostgreSQL	0,75
Diseño y funcionamiento de la API REST	0,90
Interfaz React e integración Full Stack	0,75
Pruebas automatizadas y casos de error	0,45
README, evidencias y video local	0,50
Total	5,00

Criterio transversal: una pantalla que simula resultados sin consumir la API o una API que no persiste los cambios en PostgreSQL no demuestra la integración solicitada.

10. Lista de verificación de entrega

☐ Épica, Feature, historias de usuario y tareas en Jira con sus relaciones y estados.

☐ C4 nivel 1, 2 y 3 actualizados; diagrama de clases actualizado y visible en el README.

☐ Migración o script reproducible de base de datos sin pérdida de tareas anteriores.

☐ Categorías: crear, consultar, editar y eliminar con restricción por uso.

☐ Asignación de categoría a tareas existentes y nuevas; opción Sin categoría.

☐ Filtrado de tareas desde la API REST: Todas, una categoría y Sin categoría.

☐ Mensajes de error y códigos HTTP apropiados para los casos especificados.

☐ Pruebas automatizadas del back-end y front-end con evidencia de ejecución.

☐ README completo con instrucciones de ejecución, diagramas y enlace al video.

☐ Video local que demuestra el nuevo flujo y el CRUD existente.

Lo que se evaluará: la capacidad de llevar un requerimiento nuevo desde el análisis y el diseño hasta una funcionalidad integrada, probada y documentada.

Referencia de partida

Enunciado del laboratorio ToDo: https://github.com/DOSW-2026-2/Lab6-G4-AppToDo/blob/main/README.md

Este parcial amplía los requerimientos originales del laboratorio; el mockup y las condiciones de esta evaluación son requisitos nuevos propuestos para el examen.